

Homework 1

Computer System Organization

ECE 361
Fall 2026

Overview

Homework 1 has you set up your toolchain and write a small C library for number representation and bit manipulation, the topic of Tuesday's lecture. It is released on Tue, Sep 29, 11:59pm and is due on **Sun, Oct 4, 11:59pm**, pushed to hw01/ in your course repository. All tools are allowed. After the deadline, you sit a Shine about your code, due by **Wed, Oct 7, 11:59pm**, without any supporting materials.

Homework 1 is graded *Satisfactory (S)* / *Not Yet (NY)*. It is Satisfactory when your repository meets every requirement listed at the end of this document at the deadline, and your Shine meets the bar. A Not Yet can be revised with a revision token within 3 days of receiving the grade.

Before you start: read Karumanchi 1.1–1.4 and 21.2, and come to the studio on Thursday, Oct 1, which works the same conversions by hand. The homework is much easier once you can do it on paper.

Part 1: Toolchain and repository

1. Install a C toolchain: gcc (or clang), make, git, and a bash shell. On Windows, use WSL 2 with Ubuntu. On macOS, install the Xcode command line tools (xcode-select --install). Check that gcc --version, make --version, and git --version all work.
2. Use the course repository you already created on GitHub, and enter its URL in the form on Canvas if you have not done so yet. The repository must be public so that Shine can read it.
3. Clone it and create the folders hw01/ to hw09/ and project/. Use exactly these names: Shine looks for them. Push one commit that adds your name to the README.md at the top level of the repository **by Fri, Oct 2**. If something does not work, you still have the weekend to get help in office hours.

Part 2: A bit-manipulation library

Write bits.h and bits.c with the four functions below. Use the fixed-width types from <stdint.h>. Bits are numbered from 0, the least significant bit.

Table 1. The four functions of the bit-manipulation library in Part 2.

Function	What it does
void print_binary(uint32_t x, int width)	Prints the lowest width bits of x, most significant bit first, in groups of four separated by a space. print_binary(0x2C, 8) prints 0010 1100.
uint32_t get_field(uint32_t word, int pos, int width)	Returns bits pos to pos+width-1 of word, shifted down to bit 0.
uint32_t set_field(uint32_t word, int pos, int width, uint32_t value)	Returns word with bits pos to pos+width-1 replaced by the lowest width bits of value. All other bits are unchanged.
int32_t sign_extend(uint32_t value, int width)	Interprets the lowest width bits of value as a two's complement number and returns it as an int32_t. sign_extend(0xF8, 8) returns -8.

Valid arguments are width from 1 to 32, pos from 0 to 31, and pos + width at most 32. Decide what get_field and set_field do with arguments outside that range, state it in your README, and test it. Pick a behavior you can defend. "It is undefined" is not an answer here. You work out these boundaries on paper in the studio on Thursday.

Part 3: The thermostat status word

A thermostat reports its state in a 16-bit status word:

Table 2. Layout of the 16-bit thermostat status word.

Bits	Field	Meaning
0	HEAT	1 = heater on
1	COOL	1 = compressor on
2	FAN	1 = fan on
3	FAULT	1 = fault detected
6–4	MODE	0 = OFF, 1 = HEAT, 2 = COOL, 3 = AUTO, 4 = FAN_ONLY; 5 to 7 are invalid
7	reserved	must be 0
15–8	SETPOINT	set point in °C, 8-bit two's complement (-128 to 127)

1. In `status.h` and `status.c`, define a `status_t` struct with one member per field, and write `status_t status_unpack(uint16_t word)`. Use your `get_field` and `sign_extend` from Part 2 and named constants for every position and width. No magic numbers. Watch the precedence: `==` binds tighter than `&`, so `status & 0x08 == 0` tests the wrong thing. Write `(status & 0x08) == 0`.
2. Example: `status_unpack(0x1631)` gives set point 22 °C, mode 3 (AUTO), heater on, compressor off, fan off, no fault, and the reserved bit clear. A mode of 5 to 7 is invalid: decide how your `status_t` reports it, and say so in your README.

Part 4: Build and test

The week 1 lecture slides show a Makefile pattern and a small test harness. You may start from them.

1. Write a Makefile. `make` compiles each `.c` file to its own `.o` file, using `gcc -std=c11 -Wall -Wextra` with no warnings. `make test` builds `tests/test_bits` from your objects and runs it. `make clean` removes everything the build produced.
2. Write `tests/test_bits.c`. Test every function in Part 2 at the boundaries: width 1 and 32, pos 31, a value too wide for its field, and the most negative value for `sign_extend`. Also test `status_unpack` on at least three words, including the example above. The test program prints PASS or FAIL for each test and a summary, and exits with a nonzero code if any test fails.

README and AI usage

- README.md: what your library does, how to build it and run the tests, the valid ranges of its inputs, its behavior at the boundaries, the out-of-range behavior you chose in Part 2, and how `status_unpack` reports an invalid mode.
- AI_USAGE.md: which tools you used, for what, and one thing a model got wrong or that you had to fix, and how you found it. If you used no AI, say so.

Deliverable

Push by **Sun, Oct 4, 11:59pm**. AI tools are allowed.

- top-level README.md with your name, pushed by **Fri, Oct 2**
- hw01/Makefile, bits.h, bits.c, status.h, status.c
- hw01/tests/test_bits.c
- hw01/README.md and hw01/AI_USAGE.md

Shine

The Shine window opens on Monday, Oct 5, in the morning, and your Shine is due by Wednesday, Oct 7, 11:59pm. It takes about 10 to 20 minutes. Shine builds and tests your hw01/ and asks you questions about your own code. Expect four of the five kinds of question listed in the syllabus: explain (what does this function do, and why is it written this way?), modify (change this, and tell me what breaks), expand (how

would you implement this?), and debug (why is this not working, and how would you fix it?). Transfer questions, about your project, start with homework 2.

Examples of the kind of question you may get:

- What does your `get_field` return when `width` is 32, and why is that case tricky in C?
- Change the set point to half-degree resolution in the same 8 bits. What changes, and what breaks?
- Here is a status word. Walk me through what `status_unpack` returns, and why.

HW1 Shine, due **Wed, Oct 7, 11:59pm**.

- No supporting materials: no notes, no book, no second screen, no AI, no other person.
- Webcam on. Shine records the session and notes when you leave its window.
- Do it on Monday or Tuesday, not late on Wednesday night.

Requirements for Satisfactory

- Every file in the Deliverable list above is in your repository at the deadline.
- make builds with no errors and no warnings, make test runs and all tests pass, and make clean works.
- The four functions and `status_unpack` behave as specified.
- `README.md` and `AI_USAGE.md` are present and specific to your work.
- Your Shine is completed by the deadline and meets the bar.

Looking ahead

On Tuesday, Oct 6, you receive your project instrument. Its first piece is a packed status word of its own, built with this library. Keep `bits.c` clean. You will reuse it.